Studio Nx

Backend(FastAPI/Python) 59 — create_project()

2026-10-12

Backend(FastAPI/Python) 59 — `create_project()` (프로젝트 만들기)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 새 프로젝트 만들기 (PRD 01 FR-122)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	58 `to_item` · `list_projects` · 60 `update_project`

1. 하는 일

새 프로젝트 모달에서 받은 값(56에서 검사가 끝난 값)으로 **세 행을 한 번에** 저장합니다.

표	넣는 행
`projects`	제목, 설명, BPM, 박자, 그리고 `owner_id`는 나
`project_members`	(새 프로젝트, 나, **OWNER**)
`tracks`	이름 `Master`, 종류 **MASTER**, `order_index` 0

셋 중 하나라도 실패하면 **셋 다 저장되지 않습니다** (rollback). 저장된 `Project`를 돌려주면, 라우터(62)가 `to_item`으로 응답을 만듭니다.

보낸 값	저장 결과
`{"title": " New Song "}`	제목 `New Song`, 설명 없음, BPM 120.00, 4/4로 저장되고, OWNER 행과 Master 트랙도 함께 생깁니다
`{"title": "A", "bpm": 128.5, "time_sig_num": 7, "time_sig_den": 8}`	BPM 128.50, 박자 7/8로 저장됩니다
트랙을 저장하다가 DB 오류가 남	프로젝트 행과 멤버 행도 사라지고, 오류는 그대로 위로 전달됩니다 (500)

2. 언제 쓰나

호출하는 곳	화면
`POST /api/v1/projects` 라우터 (62)	myProject.html 새 프로젝트 모달의 Create 버튼 (P1-14)

트랜잭션: 여러 SQL을 "모두 성공하거나 모두 취소하는" 한 묶음으로 처리하는 DB 기능입니다.

- `db.add`: 세션에 올려 두기만 합니다 (아직 SQL을 보내지 않습니다)
- `db.flush`: INSERT를 DB에 보냅니다. 하지만 **확정하기 전**이라서 다른 연결에서는 보이지 않고, 되돌릴 수도 있습니다
- `db.commit`: 묶음 전체를 확정합니다
- `db.rollback`: 확정하기 전의 묶음 전체를 취소합니다

3. 코드

import (파일 맨 위)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
from app.models.track import TrackType
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)

MASTER_TRACK_NAME = "Master"
```

`Track`, `MemberRole`, `TrackType`을 이번에 추가했습니다. 이 함수는 `Session`, `Project`, `ProjectMember`, `Track`, `User`, `MemberRole`, `TrackType`, `ProjectCreateRequest`, `MASTER_TRACK_NAME`을 사용합니다.

함수

```
def create_project(db: Session, user: User, data: ProjectCreateRequest) -> Project:
    project = Project(
        owner_id=user.user_id,
        title=data.title,
        description=data.description,
        bpm=data.bpm,
        time_sig_num=data.time_sig_num,
        time_sig_den=data.time_sig_den,
    )
    try:
        db.add(project)
        db.flush()  # commit 전에 project_id 를 받기 위해 (INSERT 만 먼저 보냄)
        db.add(
            ProjectMember(
                project_id=project.project_id, user_id=user.user_id, role=MemberRole.OWNER
            )
        )
        # MASTER 는 화면에서 늘 맨 아래 고정이라 순서 번호는 쓰지 않음 (2026-10-06)
        db.add(
            Track(
                project_id=project.project_id,
                name=MASTER_TRACK_NAME,
                track_type=TrackType.MASTER,
                order_index=0,
            )
        )
        db.commit()
    except Exception:
        db.rollback()
```

```
        raise
    db.refresh(project)  # DB 가 채운 created_at · updated_at · bpm 등 다시 읽기
    return project
```

문법 풀이

import 문법	뜻
`from app.models.project import MemberRole`	역할 이름 목록입니다 (OWNER, EDITOR, VIEWER). 글자를 직접 쓰는 것보다 오타가 날 위험이 적습니다
`from app.models.track import TrackType`	트랙 종류 목록입니다 (AUDIO, MIDI, MASTER, GROUP, RETURN)

식	뜻
`db.flush()` 뒤의 `project.project_id`	INSERT를 보내면 DB가 만든 자동 번호가 객체에 채워집니다. 이 번호를 멤버 행과 트랙 행에 씁니다
`try: ... except Exception: db.rollback(); raise`	어떤 오류든 먼저 취소하고, 같은 오류를 그대로 다시 던집니다 (`raise`만 쓰면 받은 오류를 그대로 던집니다)
`db.refresh(project)`	DB에서 그 행을 다시 읽습니다. `created_at`, `updated_at`처럼 DB가 채운 값을 받기 위해서입니다

4. 흐름

1. 검사가 끝난 값으로 `Project` 객체를 만듭니다.
2. `add` → `flush`로 프로젝트를 INSERT하고 번호를 받습니다.
3. 그 번호로 OWNER 멤버와 MASTER 트랙을 `add`합니다.
4. `commit`으로 남은 INSERT 두 개를 보내고, 세 행을 모두 확정합니다.
5. 2~4단계에서 오류가 나면 `rollback`으로 세 행을 모두 취소하고 오류를 다시 던집니다 (처리기가 500으로 응답합니다).
6. `refresh`로 다시 읽은 뒤 돌려줍니다.

5. 왜 이렇게

선택	이유
세 행을 한 트랜잭션으로 저장	프로젝트만 있고 OWNER 행이 없으면 `get_membership`(57)이 주인에게도 404를 냅니다. 아무도 열 수 없는 프로젝트가 생기는 것입니다. MASTER 트랙이 없으면 시퀀서가 제대로 동작하지 않습니다 (FR-122)
`flush`로 번호를 먼저 받음	멤버 행과 트랙 행에 `project_id`가 필요하지만, 아직 확정하면 안 되기 때문입니다 (템플릿 순서)
`except`에서 `rollback` 한 뒤 `raise`	실패한 세션은 rollback하기 전까지 다음 쿼리를 실행할 수 없습니다. 또 오류를 삼키지 않고 다시 던져야 처리기가 500으로 응답하고 Sentry에 기록을 남깁니다
`title.strip()`을 하지 않음	스키마(56)가 이미 공백을 지웠습니다. 그래서 템플릿 TODO에 있던 `strip()`은 필요 없어졌습니다
`MemberRole.OWNER`, `TrackType.MASTER`를 씀	글자 `"OWNER"` 대신 목록의 이름을 쓰면, 오타가 났을 때 바로 오류가 납니다

선택	이유
MASTER의 `order_index`는 0	MASTER는 트랙 목록 맨 아래에 고정되어 순서 번호를 보지 않습니다 (2026-10-06 결정). 템플릿 값을 그대로 두었습니다
`ProjectItem`이 아니라 프로젝트를 돌려줌	화면 모양으로 바꾸는 일은 `to_item`이 맡습니다. 그래서 만들기, 수정, 상세 응답이 모두 같은 모양이 됩니다

6. 시험 (`tests/test_project_service.py`: 4개를 추가해서 모두 29개)

시험	확인한 것
세 행 저장	제목의 공백이 정리되고, 설명, BPM 128.50, 7/8, `created_at` 같은 DB 값, OWNER 행, Master 트랙 하나 (MASTER, 0)가 저장됩니다
기본값	제목만 보내면 설명은 None, BPM은 120.00, 박자는 4/4입니다
목록과 응답	만든 뒤 목록에 OWNER, 멤버 1명, opened False로 나옵니다
실패하면 취소	트랙 이름을 101자로 바꿔 트랙 INSERT에서 DB 오류를 내면, 프로젝트 행과 멤버 행이 0개이고 먼저 확정한 회원은 그대로 남습니다

전체 시험 467개가 통과했고, `ruff` 검사도 통과했습니다.